Revision 50-54 · Quantization

Interactive revision of lesson 50, lesson 51, lesson 52, lesson 53 and lesson 54: each question as asked, the answer, where I slipped, and the one-line takeaway.

Q1 · 70B from two GPUs to one PARTIAL

Covers: 16, 18, 50

QUESTION

Llama 3 70B runs in BF16 across two H100s (80 GB, 3.35 TB/s each). The PM wants it on one H100 with FP8 or 4-bit weights. What's the weight memory in each format and which leaves room for KV cache, what's the idealized decode time per token, and why would FP8 on an A100 fleet cut memory but not speed up prefill?

How I solved it

1. Weights = parameters × bytes per parameter: BF16 ≈ 141 GB, FP8 ≈ 70 GB (1 byte), 4-bit ≈ 36 GB (half a byte plus scale factors).
2. FP8 fits on 80 GB but leaves only ≈ 10 GB for activations, CUDA graphs and KV cache. 4-bit leaves ≈ 40 GB, so only 4-bit really frees room for KV cache.
3. Decode reads every weight once per step, so time ≈ weight bytes ÷ bandwidth (table below).
4. The A100 has no FP8 tensor cores. FP8 weights are expanded to 16 bits before the multiply, so memory and decode improve, but prefill (limited by math) stays at BF16 speed.

Format	Weights	Decode floor	Tokens/s, one user
BF16	141 GB	$141 \times 10^9 \div 3.35 \times 10^{12} \approx 42$ ms (two GPUs in parallel ≈ 21 ms)	≈ 24
FP8	70 GB	≈ 21 ms	≈ 48
4-bit	36 GB	≈ 10.7 ms	≈ 93

4-bit on one GPU decodes about as fast as BF16 split across two GPUs.

RE-DRILL

Re-drill: check the units of every division before computing.

Q2 · Outliers, AWQ, GPTQ and SmoothQuant PARTIAL

Covers: 51, 52

QUESTION

An interviewer asks: why do a few outlier values make low-bit quantization hard, and how do AWQ, GPTQ and SmoothQuant each work around the problem?

Answer

- **Outliers:** one large value sets the group's scale, so the grid stretches to cover it and the small values round to zero. A single outlier is enough.
- **AWQ (weight-only, 4-bit):** about 1% of weight channels matter most, found by their **large activations**. AWQ multiplies those weights by a factor (e.g. ×2) before rounding so they land on a relatively finer grid, and divides the matching activations by the same factor so the output is unchanged.
- **GPTQ (weight-only):** quantizes one column at a time and pushes each rounding error onto the **weights not yet quantized**, so errors cancel instead of piling up (0.34, 0.33, 0.33 sum to 1.00 instead of 0.75). Calibration data decides how to spread the error; 175B takes about 4 GPU-hours.
- **SmoothQuant (W8A8):** the **activations** carry outliers in a few channels while weights are easy. It divides each activation channel by s and multiplies the matching weights by s, moving the difficulty into the weights. Both then fit in INT8, so the **math runs in 8 bits**: ≈ 1.5× faster and half the memory.

RE-DRILL

Re-drill: for each method, say whether it's weight-only or W8A8 and what problem it fixes.

Q3 · Why 4-bit stops paying off at big batches MISSED

Covers: 50, 51, 53

QUESTION

A 4-bit AWQ model with a Marlin kernel is about 3.5× faster than BF16 at batch 1, about 1.5× at batch 128, and roughly break-even on long prefills. Why does the gain shrink, why would a naive kernel that expands the whole matrix to 16 bits first be slower than plain BF16, and what would you switch to for a high-throughput batch workload on H100s?

Answer

- **Why the gain shrinks:** at batch 1 decode is limited by reading weights, so 4× fewer bytes gives nearly 4× (≈ 3.9× with scales). Larger batches reuse each weight for more tokens, so the limit moves from memory to math. The tensor cores still multiply in 16 bits, and each tile must also be unpacked and scaled: ≈ 1.5× at batch 128, about break-even on long prefills.
- **Why the naive kernel loses:** it reads 0.5 bytes per weight, writes 2 bytes of expanded weights, then reads those 2 bytes again: 4.5 bytes per weight versus 2 for plain BF16. A good kernel (Marlin) unpacks each tile in registers and never writes 16-bit weights back.

- **What to switch to:** **W8A8 in FP8**. Both inputs go straight into 8-bit tensor cores with no unpacking in the inner loop, and scales are applied once to the output. 8-bit math is ≈ 2× faster (1,979 vs 989 TFLOPS on H100), which speeds up the math-bound work where 4-bit runs out.

> **RE-DRILL**
>
> Re-drill: say what limits batch 1, batch 128 and prefill, then pick the format.

Q4 · "Only 0.8 points, ship it" PARTIAL

Covers: 22, 54

> **QUESTION**
>
> The 4-bit model scores 71.2% against BF16's 72.0% on 1,000 questions, and the PM says "only 0.8 points, ship it". What checks would you run before agreeing, and why isn't a matching score enough on its own?

Answer

- **Why the score isn't enough:** averages hide **flips**. Both models can score the same while some right answers turn wrong and others turn right, and users notice the ones that broke.
- **Distance:** compare the two models token by token (same top token, KL divergence). No labels needed, runs in minutes, catches broken conversions.
- **Noise floor:** run the baseline against itself, in the same engine under load. Batching alone changes answers, so only flips above this floor count.
- **Paired error bars:** on 1,000 questions each score is ±2.8 points, so the separate intervals overlap and say little. Compare question by question on the same set: 34 wrong → right, 42 right → wrong, difference −0.8 ± 1.7. It includes zero, so no significant drop, but read the 42 regressions.
- **Your product:** a few hundred real prompts with your chat template, system prompt and lengths, judged side by side.
- Decide with a rule written before the run (e.g. drop ≤ 1 point at 95% confidence, no category worse than the noise floor, product eval tie or better).

> **RE-DRILL**
>
> Re-drill: recite the ladder in order (distance, flips vs noise floor, paired accuracy, product) and say what each one catches.

Q5 · "4-bit loses 10 points" vs 98.9% recovered MISSED

Covers: 51, 54

QUESTION

A blog post says 4-bit Llama 3.1 "loses 10 points" on coding, while Red Hat's 500,000-evaluation study found 4-bit recovers 98.9% on HumanEval. What are two likely reasons for the gap, and why doesn't "98.9% recovered" mean your users get the same answers as before?

Answer

- **Mismatched chat template:** prompting the quantized model with the wrong template or system prompt hurts it for reasons unrelated to quantization. Red Hat traced many scary regressions online to this.
- **Over-sensitive eval:** strict exact-match or format checks mark a correct answer wrong when only its wording changed. Small evals add noise: a few points on a few hundred questions can be chance.
- **Recovery isn't sameness:** recovery compares average scores, and averages hide flips. Dutta et al. (NeurIPS 2024) saw accuracy move $\leq 2\%$ while up to 13.6% of answers flipped, and models with more flips were judged worse in open-ended chat. So still check flips against the noise floor and run your own prompts.

RE-DRILL

Re-drill: name two eval-setup mistakes that fake a quantization regression.